

INFORMATIQUE

2025 - 2026

Algorithmes sur les Graphes et Combinatoire

Laurent PHILIPPE

SuP_Fc

Introduction au cours AGC

Pré-requis

Ce cours est un cours d'algorithmique. Il s'appuie sur les compétences précédemment acquises dans les cours d'algorithmique pour développer des algorithmes reposant sur la théorie des graphes et pour proposer des approches algorithmiques à la résolution de problèmes d'optimisation combinatoire¹. Ainsi le cours d'algorithmique ne revient pas sur les concepts fondamentaux de l'algorithmique: structures de données simples ou complexes et structures de contrôle. Il suppose donc acquis une maîtrise de l'écriture d'algorithmes avec des structures de données telles que des matrices (tableaux à 2 dimensions), les listes, les files ou les piles.

L'écriture des algorithmes se fait dans un langage “*algorithmique*” qui n'est pas formellement défini mais qui est largement utilisé dans la communauté informatique. Il n'y a donc aucun pré-requis quant à un langage de programmation particulier.

Objectifs du cours

L'objectif de ce cours est de donner à l'étudiant qui le suit une connaissance des possibilités offertes par la modélisation à base de graphes et par différentes techniques de recherche de solutions optimales ou proches de l'optimal dans des problèmes d'optimisation.

Les compétences acquises à l'issue de ce cours sont la définition d'algorithmes sur des structures de type graphe ou arbre et l'utilisation de techniques pour résoudre des problèmes d'optimisation.

¹Vous trouverez une définition formelle du terme *optimisation combinatoire* sur [Wikipedia](#)

Recommandations

Les éléments présentés doivent être mis en application à travers les exercices donnés dans la rubrique “*Exercices*” sur moodle mais aussi en exécutant les algorithmes présentés sur des exemples. Par exemple, la plupart des algorithmes sur les graphes prennent en entrée des graphes quelconques. Il est alors facile d’exécuter l’algorithme sur un graphe simple dessiné sur une feuille. Ces exercices de déroulement permettent de mieux comprendre, donc de mieux maîtriser, les algorithmes.

Ne vous laissez pas impressionner par certains aspects mathématiques du cours, toutes les notions abordées sont à votre portée si vous êtes arrivés jusque là dans vos études. Mais si la notation vous pose problème n’hésitez pas à poser une question sur le forum.

N’hésitez pas à consulter d’autres sources d’information si les définitions et explications données dans le cours vous paraissent difficiles à comprendre: une approche différente permet parfois de résoudre ce problème.

Il est également important que vous fassiez part de vos questions et remarques dans le forum. Cela permet à l’enseignant de voir vos difficultés et aux autres étudiants de profiter des réponses qui sont données.

Écriture des algorithmes

Les algorithmes donnés dans les support sont donnés dans une écriture algorithmique, indépendante de tout langage de programmation. Dans le cours les algorithmes sont écrits de manière simplifiée, avec l’objectif de mettre en valeur les idées avant la syntaxe et de ne pas surcharger le cours. Ces algorithmes peuvent utiliser des notations mathématiques dont nous rappelons ici les principales significations.

Notations mathématiques utilisées

La définition des graphes, et des algorithmes qui les utilisent, repose en partie sur des notations mathématiques, principalement des notations ensemblistes. Nous rappelons ici la signification des notations utilisées dans le cours.

$\forall$: quelque soit

$\exists$: il existe

$\wedge$: et

$\vee$: ou

$\Rightarrow$: implique

$\neg$: négation

$\neq$: différent

∞ : infini

$\sum_{x=0}^N$: somme pour $x = 0$ jusqu'à $x = N$

$\{\dots\}$: définition d'un ensemble

$\in$: appartient

$\notin$: n'appartient pas

$\emptyset$: ensemble vide

$|V|$: cardinal de l'ensemble V

$V \times V'$: produit cartésien des ensembles V et V' , ensemble composé de tous les couples possibles constitués de deux éléments, un de V et un de V' .

$E' \subseteq E$: l'ensemble E' est un sous-ensemble, inclus dans E ou égal

$\bigcup$: union

$V \setminus \{u\}$: ensemble V moins l'élément u

Dans les exercices les algorithmes sont donnés de manière complète, c'est-à-dire, avec le détail des boucles et en utilisant explicitement les structures de données. Dans vos rendus, devoirs ou examens, il vous est demandé le même niveau de détails que celui qui est utilisé dans les exercices. Les rendus dans une version simplifiée ou dans un langage de programmation spécifique ne sont pas considérés comme corrects.

Nous donnons dans la suite les structures de données qui peuvent être utilisées dans les rendus.

Objets algorithmiques

Les algorithmes donnés dans le cours et les exercices s'appuient sur quelques objets classiques en programmation. Nous donnons ici la syntaxe utilisée (inspirée de Java) dans le cours et qui doit être utilisée dans les rendus.

Pile<type> :

- *boolean empty()*: pile vide
- *type peek()*: lit le premier élément sans le dépiler
- *type pop()*: dépile
- *push(type)*: empile

File<type> :

- *add(type)*: ajoute un élément à la fin de la file
- *boolean empty()*: file vide
- *type peek()*: lit le premier élément de la file sans le supprimer
- *type poll()*: lit et supprimer le premier élément

Liste<type> :

- *add(type)*: ajoute un élément à la fin de la liste
- *addAll(List)*: concatène la liste en paramètre à la fin de la liste
- *boolean contains(type)*: l'élément est dans la liste
- *boolean empty()*: liste est vide
- *type get(int)*: lit, sans le supprimer, l'élément dont l'index est donné en paramètre
- *type remove(index)*: supprime l'élément dont l'index est donné en paramètre
- *set(index, type)*: change l'élément dont l'index est donné en paramètre
- *int size()*: taille de la liste

Pour les listes les indexes commencent en 0.

Rappels sur les preuves

Dans certains exercices du cours il est demandé de prouver un résultat, par exemple qu'un algorithme glouton est optimal ou non.

Prouver une assertion ou une propriété revient à montrer qu'elle est vraie quelque soit le contexte. Si nous reprenons le cas d'un algorithme glouton, prouver qu'il est optimal (l'assertion est alors "l'algorithme glouton trouve toujours une solution optimale au problème") revient à prouver que, quelque soit l'instance considérée, l'algorithme nous donne une solution et qu'aucune autre solution ne peut pas donner un meilleur résultat.

À noter qu'il suffit de trouver une instance fausse pour prouver qu'une assertion est fausse mais qu'il faut prouver que toutes les instances sont justes pour prouver qu'elle est vraie.

Plusieurs approches de preuve peuvent être utilisées. Nous développons ici rapidement la preuve directe, la preuve par l'absurde ou la preuve par récurrence. Nous les illustrons par l'exemple du problème suivant: un explorateur veut remplir son sac qui peut contenir un volume donné V , à partir d'objets qui ont tous le même volume, mais pas le même poids, de manière à ce que le sac soit le plus léger possible. L'algorithme qui choisit d'abord les objets les plus légers est optimal.

Preuve directe

Dans une preuve directe, il faut prouver que quelque soit l'instance, le résultat est optimal. Une preuve directe repose, à partir de propriétés vraies, sur une suite de déductions logiques pour arriver à l'assertion finale.

Dans le cas du sac de l'explorateur il est possible de calculer toutes les sommes possibles pour constater que la somme qui contient les objets les plus légers est optimale.

Preuve par l'absurde

Dans une preuve par l'absurde nous commençons par supposer que l'assertion à prouver est fausse et nous montrons, par une suite de déductions logiques, que nous arrivons à une contradiction.

Dans le cas du sac de l'explorateur, nous prouvons l'optimalité de la solution en supposant qu'il existe une solution optimale qui contient des objets qui ne sont pas parmi les objets les plus légers. Or, dans cette solution, il est

possible de remplacer un objet plus lourd par un plus léger puisqu'ils font la même taille. Nous obtenons ainsi un sac dont le poids est inférieur à la solution initiale, donc cette solution n'est pas optimale. Nous pouvons en déduire que seule une solution avec les objets les plus légers est optimale.

Preuve par récurrence

La preuve par récurrence fonctionne sur les problèmes qui disposent d'une dimension. Nous vérifions alors que l'assertion est correcte pour la dimension 0 ou 1, nous supposons qu'elle est vraie pour une dimension n et nous utilisons cette propriété pour montrer qu'elle est vraie pour une dimension $n + 1$.

Si l'explorateur n'a la possibilité de ne mettre qu'un seul objet dans son sac alors il est évident qu'en prenant le plus léger il aura le sac le plus léger (dimension 1). Nous supposons que l'explorateur a mis les n objets les plus légers dans son sac et que celui-ci est le plus léger qu'il puisse obtenir (dimension n). Si nous considérons un sac avec une place de plus (dimension $n + 1$), alors ajouter l'objet le plus léger permet d'augmenter le moins possible le poids du sac. Puisque le sac de dimension n était le plus léger alors le sac de dimension $n + 1$ sera le plus léger puisque nous avons ajouté le moins de poids possible.

Bibliographie et webographie

Une part non négligeable de ce cours a été rédigée grâce au “*Cormen*”, livre incontournable en algorithmique dont voici la référence:

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest. *Introduction à l'algorithmique*. DUNOD. 2002.

Vous trouverez par ailleurs sur le web un grand nombre de supports de cours qui vous permettront d'élargir votre connaissance des graphes si vous le souhaitez. Attention cependant à ne pas vous éloigner du périmètre du cours en cherchant une notion. J'ai essayé de simplifier les notations pour ne pas leur donner un aspect trop rébarbatif, ce qui n'est pas toujours le cas. Il peut être tout aussi efficace de poser la question dans le forum.

Pour ceux qui apprennent mieux en vidéo diverses ressources permettent de voir d'une autre manière les notions du cours.

Pour les graphes:

https://www.youtube.com/channel/UCHtJVeNLyR1yuJ1_xCK1WRg

Pour la programmation linéaire. Un exemple qui détaille la modélisation puis la résolution graphique du problème:

<https://www.youtube.com/watch?v=60TiTsAJ22Y>

<https://www.youtube.com/watch?v=3kjTNEyjQ0E>

<https://www.youtube.com/watch?v=xKYPdYunois>